Navigation App Routing:

Finding Shortest Paths Between City Locations

Algorithm Analysis & Comparison Report

Algorithms Covered:

Dijkstra's Algorithm • A* Search • Bellman–Ford

Scenario ٦ — CS Algorithms Assignment

١. Executive Summary

> Key Finding: A* Search is the recommended primary algorithm for real-time navigation routing, leveraging geographic heuristics to reduce explored nodes by up to ٩٠٪ compared to Dijkstra on large city graphs.

This report analyzes three fundamental shortest-path algorithms in the context of real-time navigation routing (Scenario ٦). The goal is to identify which algorithm best satisfies the competing demands of speed, scalability, memory efficiency, and correctness when routing through city road networks.

Criterion	Winner	Runner-Up
Speed (typical case)	A* Search	Dijkstra
Worst-case guarantees	Dijkstra	A* Search
Negative weight support	Bellman–Ford	— (N/A for routing)
Implementation simplicity	Dijkstra	Bellman–Ford
Scalability	A* Search	Dijkstra
Real-time applicability	A* Search	Dijkstra

٢. Introduction and Problem Description

٢٫١ The Routing Challenge

Modern navigation applications such as Google Maps, Apple Maps, and Waze must compute optimal routes between arbitrary points in real time, often across graphs with millions of nodes and edges. Users expect responses within milliseconds, even during peak traffic when edge weights are changing dynamically.

This problem cannot be solved naively — enumerating all possible paths between two nodes in a city graph is computationally infeasible even for modest graph sizes. A systematic, algorithmic approach is required.

٢٫٢ Why This Problem Requires Algorithms

The navigation routing problem sits at the intersection of several competing constraints:

- Time: Routes must be computed in under a second for a good user experience.
- Scalability: Graphs can represent entire continents with millions of intersections.
- Memory: The app must run on both cloud servers and memory-limited mobile devices.
- Accuracy: Routes should be optimal (or near-optimal) to maintain user trust.
- Dynamic updates: Edge weights change continuously due to live traffic data.
- Fault tolerance: Closures, accidents, and detours must be handled gracefully.

٢٫٣ Graph Model

The road network is naturally modeled as a weighted directed graph G = (V, E, w):

- Vertices V represent road intersections (or GPS waypoints).
- Edges E represent road segments connecting intersections.
- Weight function w: E → $\mathbb{R}^+$ assigns travel time (or distance) to each road segment.
- The graph is typically sparse: each node connects to only a handful of neighbors.
- Nodes carry geographic coordinates (latitude, longitude), enabling heuristic computation.

> Scale Example: A medium city like London has roughly ١٠٠,٠٠٠ nodes and ٣٠٠,٠٠٠ edges. A country-level graph (e.g., the continental USA) can exceed ٢٤ million nodes and ٥٨ million edges.

٣. Algorithms Overview

٣,١ Dijkstra's Algorithm

Dijkstra's algorithm, published in ١٩٥٩ by Edsger W. Dijkstra, computes the single-source shortest path from a given source node to all reachable nodes in a graph with non-negative edge weights. It uses a greedy approach: always expanding the node with the currently known minimum distance.

Core Mechanism

١. Initialize all distances to infinity; set source distance to ٠.
٢. Insert source into a min-priority queue ordered by distance.
٣. Extract the minimum-distance node u; for each neighbor v, relax the edge (u,v).
٤. Repeat until the queue is empty or the destination is reached.

Pseudocode

```
function dijkstra(graph, source):
    dist[v] = INF for all v;  dist[source] = ٠
    Q = min-priority-queue;  Q.insert(source, ٠)
    while Q is not empty:
        u = Q.extract_min()
        for each edge (u, v, w) in graph.adjacency[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                Q.decrease_key(v, dist[v])
    return dist, prev
```

Key Properties

- Correctness: Guaranteed to find the shortest path for non-negative weights.
- Termination: Terminates in finite time for finite graphs.
- Optimality: Optimal among uninformed (non-heuristic) algorithms.

٣,٢ A* Search Algorithm

A* (pronounced 'A-star') is a goal-directed extension of Dijkstra's algorithm proposed by Hart, Nilsson, and Raphael (١٩٦٨). It introduces a heuristic function h(n) to guide the search toward the goal, reducing the number of nodes that must be explored for point-to-point queries.

Each node n is assigned a priority f(n) = g(n) + h(n), where:

- g(n) is the known cost from the start node to n (same as Dijkstra's dist[n]).
- h(n) is an admissible estimate of the remaining cost from n to the goal.
- For road networks: h(n) = Euclidean_distance(n, goal) / max_speed is admissible.

Pseudocode

```
function A_star(graph, start, goal, h):
    g[v] = INF for all v;  g[start] = ٠
    open = min-priority-queue ordered by f(v) = g[v] + h(v, goal)
    open.insert(start)
    while open is not empty:
        u = open.extract_min()
        if u == goal: return reconstruct_path(prev, goal)
        for each edge (u, v, w):
            tentative_g = g[u] + w
            if tentative_g < g[v]:
                g[v] = tentative_g
                prev[v] = u
                open.insert_or_decrease_key(v, g[v] + h(v, goal))
    return 'no path found'
```

Heuristic Admissibility

A heuristic h is admissible if it never overestimates the true cost: h(n) ≤ h*(n) for all n, where h*(n) is the true optimal cost. This guarantees A* finds optimal paths. A heuristic is consistent (monotone) if it satisfies the triangle inequality: h(n) ≤ w(n,m) + h(m) for all edges (n,m).

٣٫٣ Bellman–Ford Algorithm

The Bellman–Ford algorithm, independently developed by Richard Bellman (١٩٥٨) and Lester Ford (١٩٥٦), computes single-source shortest paths in graphs that may contain negative edge weights, provided there are no negative-weight cycles reachable from the source.

Pseudocode

```
function bellman_ford(graph, source):
    dist[v] = INF for all v;  dist[source] = ٠
    for i = ١ to |V| - ١:
        for each edge (u, v, w) in graph.edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
    // Detect negative cycles
    for each edge (u, v, w) in graph.edges:
        if dist[u] + w < dist[v]:
            return 'negative cycle detected'
```

```
    return dist, prev
```

Key Properties

- Handles negative edge weights (unlike Dijkstra).
- Detects negative-weight cycles (returns an error signal).
- Time complexity O(|V|·|E|) makes it impractical for large real-time graphs.

٤. Theoretical Analysis

٤,١ Complexity Summary

Let |V| denote the number of nodes (intersections) and |E| the number of edges (road segments). The three algorithms differ fundamentally in time complexity, which drives their practical applicability at scale.

Algorithm	Time (Worst Case)	Time (Typical)	Space	Negative Weights?
Dijkstra	O(|E| log|V|)	O(|E| log|V|)	O(|V| + |E|)	No
A* Search	O(|E| log|V|)	O(k log|V|), k << |E|	O(|V| + |E|)	No
Bellman–Ford	O(|V|·|E|)	O(|V|·|E|)	O(|V| + |E|)	Yes

> Practical Scale: For a medium city (|V|=١٠٠K, |E|=٣٠٠K): Dijkstra performs ~٥,١M operations; A* typically ١٠–١٠٠x fewer. Bellman–Ford requires ~٣٠ billion operations — completely infeasible for real-time use.

٤,٢ Dijkstra Complexity Analysis

Time Complexity

With a binary heap priority queue, each edge relaxation costs O(log|V|), giving:

- Total: O(|E| log|V|) — each edge is relaxed at most once.
- With a Fibonacci heap, this improves to O(|E| + |V| log|V|), optimal for dense graphs.
- In practice, binary heap implementations dominate due to lower constant factors.

Space Complexity

O(|V| + |E|) to store the adjacency list, distance array, predecessor array, and priority queue. All are linear in the input size.

٤,٣ A* Complexity Analysis

Time Complexity

A*'s worst case matches Dijkstra at O(|E| log|V|) when the heuristic is zero. In practice with a good geographic heuristic:

- The search expands a cone of nodes directed toward the goal rather than a full wavefront.
- In bidirectional A*, two simultaneous searches from source and goal further halve the explored space.
- On grid-like city graphs, a well-tuned heuristic can reduce node expansions by ٨٠–٩٥٪.

Heuristic Quality Impact

The tighter the heuristic to the true cost, the fewer nodes A* expands. Using straight-line distance / maximum road speed is both admissible and highly informative for urban road networks.

٤,٤ Bellman–Ford Complexity Analysis

Time Complexity

Bellman–Ford performs |V|-١ relaxation passes over all |E| edges: O(|V|·|E|). For our city example:

- ١٠٠,٠٠٠ × ٣٠٠,٠٠٠ = ٣٠,٠٠٠,٠٠٠,٠٠٠ (٣٠ billion) operations per query.
- At ١٠⁸ operations/second, this takes ~٣٠٠ seconds — ٣٠٠,٠٠٠x slower than Dijkstra.
- The algorithm offers no amortization benefit; each query is independently expensive.

٥. Advanced Optimisation Techniques

٥,١ Bidirectional Search

Both Dijkstra and A* can be extended to run bidirectionally: one search from the source, one from the destination, meeting in the middle. This reduces the explored search space from O(r²) to O(r²/٢) where r is the radius of the shortest-path tree.

- Bidirectional Dijkstra: standard technique used in production navigation systems.
- Bidirectional A*: more complex due to the need for consistent heuristics in both directions.

٥,٢ Contraction Hierarchies (CH)

Contraction Hierarchies is a preprocessing technique that dramatically accelerates Dijkstra and A* on static road networks. Nodes are ranked by 'importance'; less important nodes are contracted (bypassed) during preprocessing. This creates shortcut edges that encode multi-hop routes.

- Preprocessing: O(|E| log|V|) — performed once when the map is updated.
- Query time: O(|E'| log|V|) where |E'| << |E| — typically milliseconds on national graphs.
- Used by: OSRM (Open Source Routing Machine), Graphhopper, Valhalla.

٥,٣ Transit Node Routing

For very long-distance queries, transit node routing identifies a small set of highly-used nodes (e.g., major highway junctions). Any long-distance path passes through one of these transit nodes, enabling O(١) lookups for cross-country routing after preprocessing.

٥,٤ Dynamic Edge Weights and Live Traffic

Production systems update edge weights continuously from GPS probe data and traffic sensors. This requires algorithms that can efficiently recompute affected paths:

- A* and Dijkstra recompute from scratch using current weights — simple but effective.
- Dynamic Dijkstra variants (D* Lite) re-use portions of earlier computation when only a few edges change.
- Real-time traffic data is typically fused into edge weights as multiplicative congestion factors.

٥,٥ Turn Restrictions and Penalty Modeling

Real road networks include turn restrictions (no U-turns, banned left turns) and turn penalties (traffic light delays). These are modeled by expanding the graph:

- Each directed edge (u,v) is replaced by a node in the edge-based graph.

- Edges in the new graph connect pairs of original edges (u,v)→(v,w) with turn penalty weight.
- This approach integrates naturally with both Dijkstra and A* without modifying core logic.

٦. Practical Performance Analysis

٦٫١ Data Structure and Sparsity

Road networks are inherently sparse: the average intersection connects to ٢–٤ road segments. Adjacency list representation is optimal here — it uses O(|V| + |E|) memory and allows fast neighbor iteration. A* and Dijkstra both benefit directly from this structure.

Bellman–Ford iterates over all edges regardless of sparsity, negating any structural advantage.

٦٫٢ Query vs. Update Frequency

Production navigation servers face two types of events simultaneously:

- Queries: Millions of users requesting routes per minute during peak hours.
- Updates: Edge weights changing every ٣٠–٦٠ seconds as traffic conditions evolve.

Dijkstra and A* handle this naturally — each query reads current weights independently. Preprocessing-based methods (CH, Transit Nodes) must invalidate and recompute shortcuts when weights change, introducing latency on highly dynamic networks.

٦٫٣ Benchmark Estimates

Graph Scale	Dijkstra	A* (good heuristic)
Small city (١٠K nodes)	~١ ms	< ٠٫٥ ms
Medium city (١٠٠K nodes)	~٥٠ ms	١–٥ ms
Large city (٥٠٠K nodes)	~٣٠٠ ms	٥–٣٠ ms
Country-level (١٠M nodes)	~٦٠ s (too slow)	٢–١٠ s (borderline)
Country + CH preprocessing	~١ ms	< ١ ms

> Note: Estimates assume a modern server at $١٠^{٨}$ operations/second. With Contraction Hierarchies preprocessing, both Dijkstra and A* achieve sub-millisecond query times even on national-scale graphs.

٦٫٤ Memory Constraints on Mobile Devices

Mobile navigation apps must operate with limited RAM. Storing a complete national road network in memory requires several gigabytes. Common strategies include:

- Tiled maps: Load and cache only the tiles relevant to the current route.
- Compressed road data: Quantize coordinates and weights to reduce storage.
- Offline preprocessing: Distribute pre-computed CH shortcuts in the map tile format.

٧. Edge Cases and Robustness

٧٫١ Disconnected Graph Components

A road network may contain isolated components (e.g., islands reachable only by ferry). When no path exists between source and destination:

- Dijkstra terminates with dist[destination] = INF — handled gracefully.
- A* terminates when the open set is empty with no path found.
- Bellman–Ford also terminates correctly, but wastes time on all edges regardless.

٧٫٢ Dynamic Graph Mutations

Real events like road closures, accidents, and construction create sudden graph mutations:

- Edge deletion: Remove the edge; all queries computed after removal reflect the closure.
- Edge weight spike: Increase weight (e.g., ×١٠ for congestion); algorithms naturally route around it.
- Temporary restrictions: Schedule edge removal/restore with timestamp-based weight functions.

٧٫٣ Very Long-Distance Routes

For cross-country routing, even A* with a good heuristic may be slow without preprocessing. Hybrid approaches are standard:

- Use Transit Node Routing or CH for the highway backbone.
- Use A* for local first-mile and last-mile navigation.
- Hierarchical decomposition: Route at the highway level, then refine at the street level.

٧٫٤ Negative Weight Scenarios

Standard road networks use travel time or distance as weights — always non-negative. However, niche applications may encounter negative-weight analogues:

- Toll road rebates or incentive credits could create effective negative-cost edges.
- Turn cost models with aggressive reward functions may occasionally produce negative net costs.
- In these rare cases, Bellman–Ford (or Johnson's algorithm for all-pairs) is required.

٧٫٥ Heuristic Inadmissibility

If A*'s heuristic overestimates the true cost, it may return a suboptimal path. This is acceptable in practice for approximate routing under heavy load, where a route ١–٢٪ longer than optimal is indistinguishable to users but can be computed significantly faster.

٨. Trade-offs and Design Decisions

٨,١ Optimality vs. Speed

There is a fundamental tension between returning the provably optimal route and returning a very good route quickly:

Property	Dijkstra	A* Search
Guaranteed optimal?	Always	Yes (admissible h)
Approx. acceptable?	N/A	Yes (non-admissible h)
Speed trade-off	Slower, certain	Faster, configurable
Use case fit	Baseline/fallback	Production routing

٨,٢ Preprocessing vs. On-the-Fly

Algorithms split into two families:

- On-the-fly (Dijkstra, A*): No preprocessing required. Work with dynamic weights. Slower on large graphs without assistance.
- Preprocessing-based (CH, TNR): Extremely fast queries. Require rebuild when the base graph changes. Best for semi-static networks.

For production systems, the standard architecture combines both: preprocessing on the static road skeleton, A* for real-time adjustments.

٨,٣ Implementation Complexity

Algorithm	Implementation Effort	Library Availability
Dijkstra	Low — well documented	Excellent (NetworkX, Boost, etc.)
A* Search	Medium — heuristic design required	Good (most routing libraries)
Bellman–Ford	Low — simple iteration	Excellent, but rarely used for routing
Contraction Hierarchies	High — complex preprocessing	Moderate (OSRM, Graphhopper)

٩. Recommendation and Conclusion

٩,١ Primary Recommendation: A* Search

> Recommendation: Use A* Search as the primary routing algorithm for point-to-point city navigation, with Euclidean-distance / max-speed as the admissible heuristic.

A* is the clear winner for the navigation routing scenario for the following reasons:

- It leverages the geographic embedding of road networks — straight-line distance is a natural, admissible, and highly informative heuristic.

- It dramatically reduces the number of explored nodes (typically ٨٠–٩٥٪ fewer than Dijkstra on city graphs) without sacrificing correctness.
- It gracefully handles dynamic edge weights — each query re-evaluates from current weights.
- It is well-supported by open-source libraries and is the basis of virtually all production navigation routing engines.

٩٫٢ Secondary Recommendation: Dijkstra as Baseline

Dijkstra's algorithm should be maintained as:

- A correctness baseline for testing and benchmarking A* implementations.
- A fallback when a suitable heuristic is unavailable (e.g., non-geographic routing).
- The foundation for Contraction Hierarchies preprocessing pipelines.

٩٫٣ Bellman–Ford: Reserve for Specialized Cases

Bellman–Ford should not be used for real-time routing. Its O(|V|·|E|) complexity renders it ٣٠٠,٠٠٠× slower than Dijkstra on medium city graphs. Reserve it exclusively for:

- Graphs with genuine negative edge weights (e.g., financial arbitrage models).
- Negative-cycle detection as a validation step after graph construction.
- Educational contexts where its generality is the pedagogical point.

٩٫٤ Scaling to Production

For systems scaling from individual cities to national or global coverage, the recommended architecture is:

٥. Build a Contraction Hierarchy on the static road graph during off-peak preprocessing.
٦. Use A* over the contracted graph for millisecond-level queries.
٧. Apply dynamic edge weight updates to the underlying graph; invalidate shortcuts as needed.
٨. Deploy bidirectional A* for further speed improvements on long-distance queries.
٩. Tile the map for mobile devices; run A* over in-memory tiles with seamless boundary crossing.

٩٫٥ Final Comparison Matrix

Algorithm	Time Complexity	Handles Neg. Weights	Recommended Use	Verdict
Dijkstra	O(|E| log|V|)	No	Baseline, preprocessing	✓ Secondary
A* Search	O(k log|V|) typical	No	Real-time routing	★ Primary
Bellman–Ford	O(|V|·|E|)	Yes	Neg. weights / research	✗ Not for routing

> Final Word: A* Search, backed by Contraction Hierarchies for scale, represents the state of the art for production navigation routing. It is what powers Google Maps, Apple Maps, and the major open-source routing engines today.

١٠. References

Dijkstra, E. W. (١٩٥٩). A note on two problems in connexion with graphs. Numerische Mathematik, ١(١), ٢٦٩–٢٧١.

Hart, P. E., Nilsson, N. J., & Raphael, B. (١٩٦٨). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, ٤(٢), ١٠٠–١٠٧.

Bellman, R. (١٩٥٨). On a routing problem. Quarterly of Applied Mathematics, ١٦(١), ٨٧–٩٠.

Geisberger, R., Sanders, P., Schultes, D., & Delling, D. (٢٠٠٨). Contraction hierarchies: Faster and simpler hierarchical routing in road networks. Experimental Algorithms, ٣١٩–٣٣٣.

Bauer, R., & Delling, D. (٢٠٠٩). SHARC: Fast and robust unidirectional routing. Journal of Experimental Algorithmics, ١٤, ٢,٤.

OSRM Project. (٢٠٢٤). Open Source Routing Machine — Technical Documentation. https://project-osrm.org